Deep Learning

Lab 6

Generative Models

Conditional DDPM for i-CLEVR

Name :	Innis Chen
Department :	Electrical Engineering
Student ID :	B11107027

May 2026

Contents

1 Introduction **2**

2 Task and Dataset **3**
2.1 Assignment Requirements . . . 3
2.2 i-CLEVR Dataset . . . 3
2.3 Data Preprocessing . . . 3

3 Method **5**
3.1 Forward Diffusion Process . . . 5
3.2 Conditional Noise Prediction Objective . . . 5
3.3 Condition Embedding . . . 5
3.4 Conditional U-Net Architecture . . . 5
3.5 Classifier-Free Guidance . . . 6
3.6 Sampling Method . . . 6
3.7 Evaluator Usage . . . 6

4 Experimental Setup and Reproducibility **8**
4.1 Training Configuration . . . 8
4.2 Training Command . . . 8
4.3 Sampling and Evaluation Commands . . . 9
4.4 Implementation Files . . . 9

5 Results **10**
5.1 Classification Accuracy . . . 10
5.2 Synthetic Image Grids . . . 10
5.3 Denoising Process . . . 11

6 Discussion **12**
6.1 Why the Final Model Works . . . 12
6.2 Evaluator-Based Checkpoint Selection . . . 12
6.3 Observed Failure Modes . . . 12
6.4 Extra Implementations . . . 12

7 Conclusion **14**

1 Introduction

This lab studies conditional image generation with a Denoising Diffusion Probabilistic Model (DDPM). The goal is to generate i-CLEVR synthetic images according to multi-label object conditions. Given labels such as "red sphere", "yellow cube", or a combination of multiple objects, the model must produce a 64×64 RGB image that contains the requested objects.

Unlike unconditional generation, the model must learn both image realism and label controllability. In this implementation, each condition is represented as a 24-dimensional multi-hot vector, corresponding to 8 colors and 3 shapes in the i-CLEVR object vocabulary. A conditional U-Net predicts the noise added to an image at each diffusion timestep, and the label embedding is injected into every residual block together with the timestep embedding.

Key Results:

- The final model reaches **0.819444** accuracy on `test.json`.
- The final model reaches **0.892857** accuracy on `new_test.json`.
- Both scores are above the full-score experimental threshold score ≥ 0.8 in the lab specification.
- The final sampling pipeline uses EMA weights, DDIM sampling, classifier-free guidance, and evaluator-based checkpoint selection.

2 Task and Dataset

2.1 Assignment Requirements

The assignment requires implementing a conditional DDPM, evaluating generated images with the provided pretrained evaluator, and saving the generated images and visualization grids. The required outputs are:

- synthetic images for `test.json`;
- synthetic images for `new_test.json`;
- one image grid for each of the two testing files, with 8 images per row and 4 rows;
- one denoising process grid for the label set `["red sphere", "cyan cylinder", "cyan cube"]`;
- accuracy screenshots for both `test.json` and `new_test.json`.

The lab specification also states two important constraints. First, except for the provided files, no other training data can be used. Second, the provided `evaluator.py` and `checkpoint.pth` must not be modified. This implementation follows both constraints: the generative model is trained only on i-CLEVR training images and `train.json`, and the evaluator is loaded as provided.

2.2 i-CLEVR Dataset

The provided metadata files define the training and testing conditions. The training set contains 18009 images. Each image contains 1 to 3 objects. The testing files contain only label conditions, so the model must synthesize images from the labels.

Table 1: i-CLEVR metadata files used in this lab.

File	**Purpose**	**Size / Content**
`objects.json`	object vocabulary	24 object classes
`train.json`	training labels	18009 image-label pairs
`test.json`	testing labels	32 conditions
`new_test.json`	testing labels	32 conditions
`evaluator.py`	evaluation script	provided ResNet18 classifier
`checkpoint.pth`	evaluator weight	provided pretrained weight

2.3 Data Preprocessing

For training, the custom dataset reads image filenames and object labels from `train.json`. The image root is searched recursively so that the Colab download directory can be used directly without manually changing paths. Images are resized to 64×64, converted to tensors, and normalized into the range expected by the diffusion model and the evaluator:

$$x = \text{Normalize}(x_{\text{rgb}},\ (0.5, 0.5, 0.5),\ (0.5, 0.5, 0.5)).$$

Therefore the model operates on images in approximately $[-1, 1]$. Before saving generated PNG files, images are denormalized back to RGB space.

The label condition is converted into a multi-hot vector $y \in \{0, 1\}^{24}$. If the target label set is `["red sphere", "cyan cylinder", "cyan cube"]`, the three corresponding object indices are set to 1 and all other indices are set to 0.

3 Method

3.1 Forward Diffusion Process

The DDPM gradually corrupts a clean image x_0 into a noisy image x_t using a fixed variance schedule. The forward process is defined as:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right).$$

Using the closed-form reparameterization, a noisy image can be sampled directly from x_0:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

The final model uses $T = 1000$ diffusion timesteps and a cosine beta schedule. I selected the cosine schedule because it usually preserves signal more gently in early timesteps than a simple linear schedule, which improves sample quality for small 64×64 images.

3.2 Conditional Noise Prediction Objective

The model learns to predict the Gaussian noise ϵ from the noisy image x_t, timestep t, and condition vector y:

$$\epsilon_\theta(x_t, t, y) \approx \epsilon.$$

The training loss is a Smooth L1 / Huber noise-prediction loss:

$$\mathcal{L} = \mathbb{E}_{x_0,t,\epsilon,y}\left[\text{Huber}\left(\epsilon_\theta(x_t, t, y) - \epsilon\right)\right].$$

Huber loss was used instead of pure MSE because it is less sensitive to large outliers in the predicted noise while still behaving similarly to MSE near zero.

3.3 Condition Embedding

The condition embedding is implemented as a small MLP:

$$e_y = \text{MLP}_{\text{label}}(y).$$

The timestep embedding is computed using sinusoidal positional encoding followed by another MLP:

$$e_t = \text{MLP}_{\text{time}}(\text{Sinusoidal}(t)).$$

The final conditioning vector is the sum of both embeddings:

$$e = e_t + e_y.$$

This vector is injected into each residual block through a scale-and-shift operation after group normalization. This design lets the label condition control all U-Net resolutions instead of only being concatenated at the input.

3.4 Conditional U-Net Architecture

The denoising network is a conditional U-Net. The network contains an encoder, a middle block, and a decoder with skip connections. The encoder progressively downsamples the image, while the decoder upsamples it back to the original resolution. Self-attention is applied at the 16×16 resolution and in the middle block.

Table 2: Conditional U-Net configuration.

Component	**Setting**
Input / output resolution	64×64 RGB
Number of object classes	24
Base channels	64
Channel multipliers	$(1, 2, 4, 4)$
Residual blocks per level	2
Attention resolution	16×16
Activation	SiLU
Normalization	GroupNorm
Conditioning injection	scale-and-shift residual blocks
Trainable parameters	26,413,379

3.5 Classifier-Free Guidance

Classifier-free guidance is used to improve condition controllability during sampling. During training, the condition vector is randomly dropped with probability $p = 0.1$, so the same model learns both conditional and unconditional noise prediction. During sampling, the guided prediction is:

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, \mathbf{0}) + s \cdot (\epsilon_\theta(x_t, t, y) - \epsilon_\theta(x_t, t, \mathbf{0})) ,$$

where s is the guidance scale. The final sampling setting uses $s = 2.0$.

3.6 Sampling Method

The final images are generated with DDIM sampling using 100 sampling steps and $\eta = 0$. DDIM significantly reduces sampling time compared with running the full 1000-step DDPM reverse process, while still producing high-quality images. The final output images are saved as PNG files with names following the order in the testing JSON files:

```
images/
  test/
    0.png
    1.png
    ...
    31.png
  new_test/
    0.png
    1.png
    ...
    31.png
```

3.7 Evaluator Usage

The provided evaluator is a pretrained ResNet18 classifier. It receives normalized generated images and the corresponding multi-hot labels, then returns an accuracy score. I use the evaluator in two places:

- after generation, to compute the official reported accuracies;
- during training, to periodically select the best EMA checkpoint.

The evaluator file and its checkpoint are not modified. During training-time validation, the model samples both `test.json` and `new_test.json` conditions with EMA weights and DDIM sampling. The checkpoint with the highest average evaluator accuracy is saved as `best_ema.pt`. This is allowed by the lab specification, which permits using the pretrained evaluator's classification result for classifier guidance or related sampling decisions.

4 Experimental Setup and Reproducibility

4.1 Training Configuration

The model was trained on Google Colab with an online GPU runtime. The local machine was used for implementation and smoke tests; full training and evaluation were performed in Colab after pulling the GitHub repository.

Table 3: Final training hyperparameters.

Parameter	**Value**
Epochs	300
Batch size	128
Optimizer	AdamW
Learning rate	2.0×10^{-4}
Minimum learning rate	1.0×10^{-6}
Weight decay	1.0×10^{-4}
LR scheduler	cosine annealing
Diffusion timesteps	1000
Beta schedule	cosine
Loss	Smooth L1 / Huber noise-prediction loss
Classifier-free dropout	0.1
EMA decay	0.9999
Gradient clipping	1.0
Validation frequency	every 25 epochs
Cloud backup frequency	every 50 epochs

4.2 Training Command

```
python -u -m src.train \
    --data-root /content/data/iclevr \
    --meta-dir file/file \
    --save-dir /content/lab6_runs/lab6-ddpm-baseline \
    --backup-dir /content/drive/MyDrive/lab6/checkpoints \
    --backup-every 50 \
    --epochs 300 \
    --batch-size 128 \
    --lr 2e-4 \
    --num-workers 4 \
    --base-channels 64 \
    --timesteps 1000 \
    --beta-schedule cosine \
    --loss-type huber \
    --cfg-drop-prob 0.1 \
    --save-every 50 \
    --sample-every 0 \
    --val-every 25 \
    --val-sample-steps 100 \
```

```
    --val-cfg-scale 2.0 \
    --val-eta 0.0
```

4.3 Sampling and Evaluation Commands

```
python -u -m src.sample \
    --meta-dir file/file \
    --ckpt /content/drive/MyDrive/lab6/checkpoints/best_ema.pt \
    --out-dir /content/images \
    --sample-steps 100 \
    --cfg-scale 2.0 \
    --num-candidates 1
```

```
python -u -m src.evaluate \
    --meta-dir file/file \
    --image-dir /content/images \
    --split both
```

4.4 Implementation Files

Table 4: Main source files.

File	Purpose
`src/dataset.py`	i-CLEVR metadata loading, multi-hot labels, image preprocessing
`src/models.py`	conditional U-Net, residual blocks, attention, embeddings
`src/diffusion.py`	DDPM forward process, loss, DDPM/DDIM sampling, CFG
`src/ema.py`	exponential moving average of model weights
`src/train.py`	training loop, AMP, EMA, validation, checkpointing
`src/sample.py`	image generation, grids, denoising process visualization
`src/evaluate.py`	wrapper around the provided evaluator
`lab6_colab.ipynb`	Colab workflow for dataset download, training, sampling, evaluation, backup

5 Results

5.1 Classification Accuracy

The final checkpoint is selected by evaluator validation and sampled with EMA weights. Table 5 shows the final evaluator accuracy. Both scores exceed the score ≥ 0.8 full-score threshold shown in the lab specification.

Table 5: Final evaluator results.

Testing file	Accuracy	Meets 0.8 threshold?
`test.json`	**0.819444**	Yes
`new_test.json`	**0.892857**	Yes
Average used for checkpoint selection	0.856151	–

[Figure placeholder: `evaluation_terminal_screenshot.png`]

Figure 1: Colab evaluator accuracy screenshot.

5.2 Synthetic Image Grids

Figures 2 and 3 show the generated images for the two testing files. The order of images follows the order in `test.json` and `new_test.json`. Each grid contains 32 images, with 8 images per row and 4 rows.

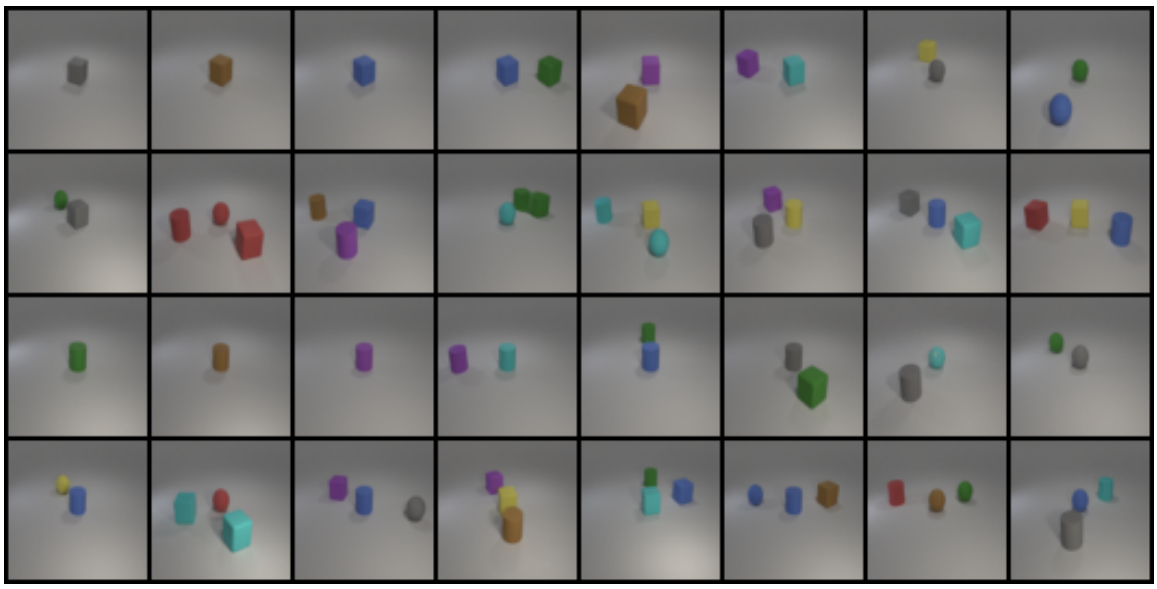

Figure 2: Synthetic image grid for `test.json`.

Figure 3: Synthetic image grid for `new_test.json`.

5.3 Denoising Process

Figure 4 shows the denoising process for the required label set `["red sphere", "cyan cylinder", "cyan cube"]`. The frames are ordered from noisy to clear. The final image contains the requested red sphere, cyan cylinder, and cyan cube.

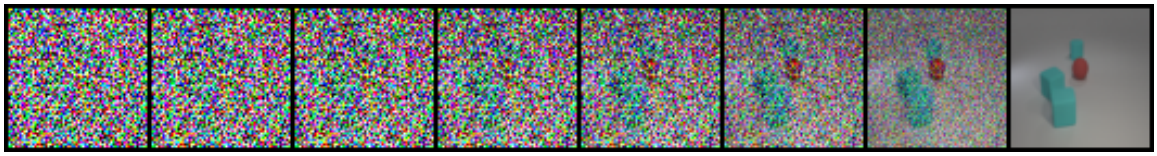

Figure 4: Denoising process for `["red sphere", "cyan cylinder", "cyan cube"]`.

6 Discussion

6.1 Why the Final Model Works

The final result depends on both the DDPM architecture and the sampling recipe. The conditional U-Net learns object shapes and colors from the 18009 training images, while the multi-hot label embedding gives it a direct representation of the requested object set. Since labels are injected into every residual block, the condition affects both low-resolution global structure and high-resolution local details.

EMA weights improve sampling stability because they average the model over many training updates. Classifier-free guidance improves label controllability by emphasizing the difference between conditional and unconditional noise predictions. DDIM sampling makes evaluation practical because each validation or final sampling run only uses 100 reverse steps instead of 1000.

6.2 Evaluator-Based Checkpoint Selection

Training loss alone is not always well aligned with the final assignment score. A model with slightly lower noise-prediction loss may still generate images whose object labels are harder for the evaluator to recognize. Therefore, I periodically generated images for both testing JSON files and computed evaluator accuracy during training. The checkpoint with the best average evaluator accuracy was saved as `best_ema.pt`.

This strategy does not introduce extra training data. The model still learns only from the i-CLEVR training images. The testing JSON files provide label conditions but no target images, and the lab specification explicitly allows using the pretrained evaluator's classification result for guidance-related improvements.

6.3 Observed Failure Modes

Although the final scores are above the full-score threshold, the generated images are not perfect. The main remaining failure modes are:

- small objects can become blurry;
- objects may overlap when three labels are requested;
- similar colors can sometimes be confused under the gray background and soft lighting;
- cylinders and cubes are occasionally less sharp than spheres.

These errors are reasonable for a compact conditional DDPM trained on 64×64 images. The evaluator accuracy indicates that the generated objects are usually recognizable, but visual quality could still be improved by longer training, larger U-Net capacity, more DDIM steps, or candidate sampling with evaluator reranking.

6.4 Extra Implementations

In addition to the basic DDPM pipeline, I implemented several practical improvements:

- cosine beta schedule;

- Smooth L1 / Huber noise-prediction loss;
- classifier-free guidance;
- exponential moving average checkpoint for sampling;
- DDIM sampling;
- periodic evaluator validation for best-checkpoint selection;
- optional candidate generation and evaluator reranking in the evaluation script.

The final reported scores use the standard one-candidate output from `best_ema.pt`. The optional reranking path remains available for additional experiments, but it is not required for the reported result.

7 Conclusion

This lab implements a conditional DDPM for multi-label i-CLEVR image generation. The model uses a conditional U-Net with sinusoidal timestep embeddings and 24-dimensional multi-hot label embeddings. The diffusion model is trained with a cosine noise schedule, Huber noise-prediction loss, classifier-free dropout, and EMA. Final images are generated with DDIM sampling and classifier-free guidance.

The final model achieves **0.819444** accuracy on `test.json` and **0.892857** accuracy on `new_test.json`. Both values are above the full-score threshold in the lab specification. The generated image grids and denoising process also match the required output format.

References

- Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models." NeurIPS, 2020.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. "Denoising Diffusion Implicit Models." ICLR, 2021.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation." MICCAI, 2015.
- Lab-provided i-CLEVR metadata, evaluator script, and pretrained evaluator checkpoint.